Sumo: Dynamic and Generalizable Whole-Body Loco-Manipulation

John Z. Zhang[1,2], Maks Sorokin[2]*, Jan Brüdigam[2]*, Brandon Hung[2]*, Stephen Phillips[2], Dmitry Yershov[2], Farzad Niroui[2], Tong Zhao[2], Leonor Fermoselle[2], Xinghao Zhu[2], Chao Cao[2], Duy Ta[2], Tao Pang[2], Jiuguang Wang[2], Preston Culbertson[2,3], Zachary Manchester[1], and Simon Le Cléac'h[2]
[1]MIT, [2]RAI Institute, [3]Cornell University, *Equal Contribution.
This work was done in part during an internship at the RAI Institute.
Corresponding Email: jzhang3@mit.edu.

arXiv:2604.08508v3 [cs.RO] 9 Sep 2026

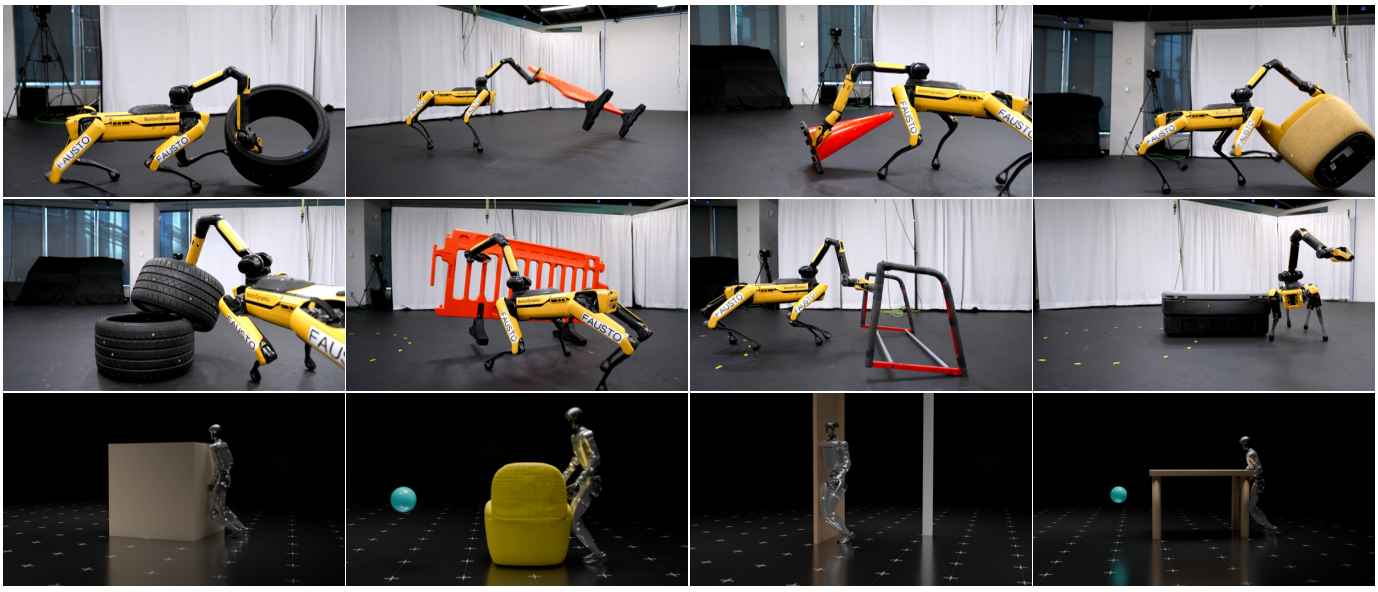

Fig. 1: First row from left to right: Spot robot uprights a tire, crowd-control barrier, traffic cone, and chair. Second row from left to right: Spot robot stacks a tire on top of another tire, drags a crowd-control barrier, a tire rack, and large box to a target position. Third row from left to right: simulated G1 humanoid robot pushes a box, chair, door, and a dining table.

***Abstract*—This paper presents a sim-to-real approach that enables legged robots to dynamically manipulate large and heavy objects with whole-body dexterity. Our key insight is that by performing test-time steering of a pre-trained whole-body control policy with a sample-based planner, we can enable these robots to solve a variety of dynamic loco-manipulation tasks. Interestingly, we find our method generalizes to a diverse set of objects and tasks with no additional tuning or training, and can be further enhanced by flexibly adjusting the cost function at test time. We demonstrate the capabilities of our approach through a variety of challenging loco-manipulation tasks on a Spot quadruped robot in the real world, including uprighting a tire heavier than the robot's nominal lifting capacity and dragging a crowd-control barrier larger and taller than the robot itself. Additionally, we show that the same approach can be generalized to humanoid loco-manipulation tasks, such as opening a door and pushing a table, in simulation. Project code and videos are available at https://sumo.rai-inst.com/.**

I. Introduction

Humans and animals physically interact with the world in creative ways: Their ability to gracefully make and break contact with their environment enables them to traverse treacherous terrain and move objects several times larger than themselves. Achieving similar levels of athletic intelligence for robotic systems has been a long-standing challenge. Over the past decade, developments in numerical optimization [10, 12, 5] and machine learning [15, 8] have made significant progress towards building robots with agility and dexterity [42, 36]. The next research frontier lies in enabling robots to manipulate objects during locomotion, or so-called loco-manipulation. So far, research on loco-manipulation has largely focused on learning from human demonstrations through teleoperation or video imitation, which are usually limited to quasi static table-top settings and fails to leverage the passive dynamics of the object or the robot. Algorithms and systems that enable a robot to autonomously coordinate its entire body to dynamically move large and heavy objects during loco-manipulation have, so far, been elusive.

This paper studies loco-manipulation problems in which the manipulated object is large, heavy, or geometrically complex enough that success requires dynamic coordination of the robot's whole body. Recent works have shown impressive in-the-wild locomotion agility enabled by reinforcement learning (RL). However, adapting RL to each new manipulation task often requires substantial reward engineering, retraining, and

compute, and can be difficult to generalize beyond the training distribution. On the other hand, sample-based model-predictive control (MPC) has proven to be a simple and training-free method for contact-rich manipulation [14, 1, 25, 24]. Yet sample-based MPC struggles to find good solutions for high-degree-of-freedom robots or dynamically unstable tasks, which frequently occur in quadruped and humanoid loco-manipulation.

Our approach combines the strengths of both. We start from a pre-trained generalist whole-body control policy learned with RL and use real-time sample-based MPC to steer it for contact-rich manipulation of previously unseen objects. This hierarchy addresses key shortcomings of both end-to-end RL and sample-based MPC while preserving their main advantages. RL provides a robust whole-body controller when the training distribution can be modeled with sufficient randomization [55]. In turn, the low-level policy reduces the effective action space for online planning and stabilizes the dynamics, mitigating the divergence issues associated with single-shooting rollouts of unstable systems.

Our framework, which we call Sumo, enables the high-level sample-based controller to steer the policy effectively for real-world manipulation. Our experiments reveal two interesting properties of this approach. First, hierarchical structure simplifies loco-manipulation. Compared to end-to-end MPC, planning in the command space of a pre-trained whole-body policy reduces the effective search space and stabilizes the robot's contact-rich dynamics. Compared to end-to-end RL, the same hierarchy achieves similar or better success with much simpler task objectives and without task-specific retraining. Second, planning enables generalization. Here, we define generalization as reusing the same controller on new objects and new task objectives by changing only the planner's object model or cost function at test time, without retraining. Keeping high-level decision making online allows the same framework to make these adaptations directly at deployment. Finally, our approach is also ammenable to desiging complex behaviors via reward shaping more quickly by avoiding offline policy training. The result is a robust yet flexible framework that enables quadruped and humanoid robots to use their entire body to manipulate objects with complex geometries and with sizes and weights comparable to the robots themselves, Fig. 1.

Our contributions include:

- A hierarchical framework that combines a pre-trained generalist whole-body control policy and test-time planning for dynamic whole-body loco-manipulation.
- A set of experimental comparisons showing that hierarchical structure simplifies loco-manipulation by reducing search difficulty and task-specification complexity relative to end-to-end MPC and end-to-end RL baselines.
- A set of experimental comparisons showing that test-time planning enables adaptation to new objects and new task objectives without retraining by changing the object model or cost function at deployment.
- A set of eight real-world demonstrations on the Spot quadruped robot and four demonstrations on the G1 humanoid robot in simulation covering diverse and challenging loco-manipulation scenarios.
- An open-source benchmark and dataset focusing on loco-manipulation tasks with objects that require whole-body coordination due to their size and weight relative to the robots' physical limits.

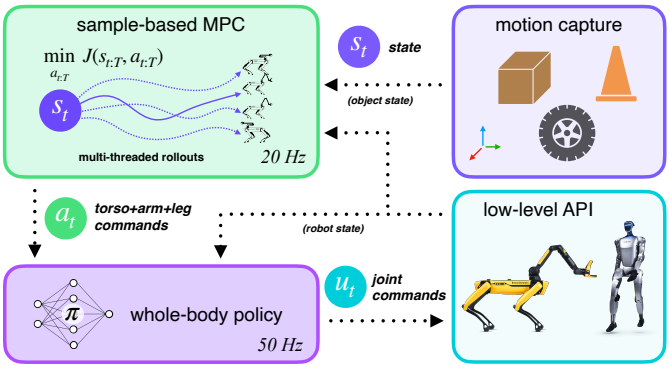

Fig. 2: System overview: Our method takes a hierarchical approach that combines a pre-trained whole-body control (WBC) policy (purple) with high-level sample-based MPC (green). The low-level whole-body control policy takes in the current state and desired torso, arm, and leg commands and outputs the joint-level commands for the quadruped or humanoid robot at 50Hz. The high-level sample-based MPC aims to minimize a task-specific cost function by taking in the current state estimate and solving for the desired torso, arm, and leg commands for the low-level policy at 20Hz.

The remainder of this paper reviews related work (Sec. II), details the Sumo framework (Sec. III), analyzes it against baselines in controlled simulation (Sec. IV), demonstrates it on a real-world Spot quadruped and a simulated G1 humanoid (Sec. V), and concludes with limitations and future directions (Sec. VI).

II. Background and Related Works

This section reviews relevant literature in RL, sample-based MPC, and their recent applications to loco-manipulation.

A. Reinforcement Learning for Whole-Body Control

Deep reinforcement learning aims to learn neural-network policies via self-play trial and error. In robotics, the advancements in large-scale parallel simulation platforms [31, 11] on accelerated hardware (such as GPUs) have significantly reduced the cost of collecting data in simulation. As a result, on-policy RL algorithms, such as PPO [38], have become a staple in the sim-to-real policy optimization paradigm. Additionally, RL naturally incorporates robust policy optimization via domain randomization techniques [43], reducing the so-called sim-to-real gap.

Despite its promise, this RL paradigm faces several significant drawbacks in practice: First, RL typically requires dense reward design to find efficient policies with finite compute [34, 2]. Second, the final policy is often sensitive to reward design, and finding a suitable reward term itself is often a non-trivial and manual search problem. Because of these practical shortcomings, researchers typically have to go

through a time-consuming loop of reward engineering, policy training, and hardware testing, before going back to reward engineering for each task. While the recent emergence of LLMs [51, 30] can automate this process to some degree, reliably finding suitable rewards that lead to successful sim-to-real transfer still requires (expert) human-in-the-loop intervention, especially when policy optimization is time-consuming.

The sim-to-real RL paradigm has been particularly successful for legged robot locomotion [15, 8], where rewards have become relatively standardized and one can carefully design curricula for difficult terrain variations. However, in open-world manipulation, robots are faced with reasoning over a diverse set of scenarios including objects that vary in size, weight, geometry, friction, etc. This test-time diversity makes randomizing all possible distributions challenging at training time for sim-to-real RL. As a result, successful real-world manipulation, so far, has come from learning from demonstrations [9].

In this work, we take advantage of RL's strength—learning robust locomotion policies over complex terrains offline—and leave the complexity of generalizable contact-rich decision making to online search via sample-based MPC.

B. Sample-Based MPC

Sample-based MPC is a family of zeroth-order trajectory-optimization methods that aims to solve for an optimal control sequence over a (typically short) prediction horizon. Sample-based MPC has been derived from many perspectives, including path integrals [46], information theory [47], importance sampling [54], and diffusion [35]. Additionally, sample-based MPC methods are closely related to other derivative-free optimization algorithms such as CMA-ES [13]. Notably, similar to zeroth-order RL, which typically requires lots of compute *offline*, sample-based MPC can be viewed as an *online* policy-gradient method [37] that is entirely training-free, making it extremely flexible at test time.

Because of its parallel nature and ease of implementation, this family of trajectory optimization algorithms has grown in popularity in recent years, along with the rise of modern parallel computing hardware. The derivative-free nature of sample-based MPC is amenable to contact-rich dynamics and learned black-box deep neural network dynamics models, where computing derivatives is expensive or unreliable [45, 53], which often occurs in contact-rich control. In contrast, classical gradient-based methods such as the iterative linear-quadratic regulator (iLQR) [16, 26] and direct-collocation [21] typically struggle in these scenarios without additional care to modify the derivatives [10].

Online sample-based MPC has two main pitfalls: the curse of dimensionality and unstable dynamical systems. As an online search algorithm, sample-based MPC suffers from the curse of dimensionality that stems from high-dimensional input spaces associated with highly articulated robots and longer prediction horizons. Recent works have proposed to reduce the temporal dimensionality by sampling over low-order spline control points [14, 25, 1], making sample-based MPC tractable in real-time for robots like dexterous hands, quadrupeds, and humanoids. Furthermore, sample-based MPC generally relies on single-shooting dynamics rollouts, leading to divergence on open-loop unstable systems, which commonly occur in legged robotics.

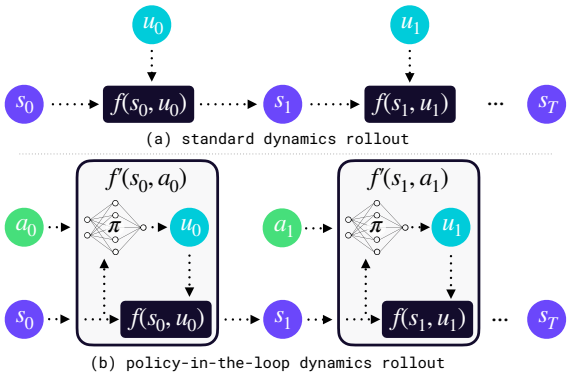

Fig. 3: Illustrations comparing (a) standard dynamics rollouts f where the actions u are the joint-level controls for the multi-body dynamics model and (b) our policy-in-the-loop dynamics rollouts f', where the actions a are inputs to the low-level locomotion policy π.

In our framework, by leveraging a low-level WBC policy, the sample-based MPC enjoys the benefit of sampling in a reduced action space on a stabilized dynamical system, drastically improving sample efficiency during online search.

C. Loco-Manipulation

Building robots that manipulate objects during locomotion, or so-called "loco-manipulation", has become an increasingly important yet challenging research direction as it inherits the fundamental challenges of both manipulation and locomotion for legged robots. Prior works have deployed both model-based methods [1] and RL methods [39] to achieve joint locomotion and manipulation of objects. Notably, several works have investigated using legs as manipulators [55, 7, 3, 41, 6].

For humanoid loco-manipulation, retargeting human motions has become a popular method to bootstrap downstream learning and optimization [48, 52]. However, these teleoperation and retargeting approaches are much less applicable to non-anthropomorphic robots, like Spot. [40] explores building a planning and control framework for legged robot loco-manipulation, but the control module requires fixed contact modes during planning and is limited to quasi-static behaviors.

Different from existing literature, we focus on manipulating objects that are larger than the robot itself or heavier than the maximum lifting capacity of the robot, thus requiring the robot to efficiently coordinate its entire body to solve these tasks. Our method naturally incorporates RL and online MPC while retaining detailed robot dynamics and collision geometry, and without requiring fixed contact modes during planning [6, 40].

D. Hierarchical Control Methods

Many recent works aim to leverage the robustness of RL policies together with the structure of model-based control [19, 6, 49, 20]. This broad family includes RL-augmented whole-body controllers, optimization-guided tracking policies, and residual policies around MPC controllers. These methods have

demonstrated strong tracking, locomotion, and force-control capabilities by using model-based components to provide structure, reference motions, or low-level constraints while learned policies improve robustness. Sumo is closely related to this line of work, but studies a complementary setting: autonomous, goal-conditioned loco-manipulation in which no teleoperator, expert motion library, or task-specific model-based planner provides a reference motion. In this setting, the online optimizer must reason over object interactions and manipulation objectives directly.

More broadly, a common design pattern in hybrid RL-MPC for contact-rich robotics is a high-level RL, low-level MPC hierarchy [19, 6, 49, 20]. Sumo instead explores the alternative high-level MPC, low-level RL paradigm, motivated by the recent emergence of powerful generalist tracking policies [55, 27, 29] that typically interface with task-space teleoperation or follow motion references. By using MPC to steer such a policy online, we can synthesize dynamic behaviors from task objectives at deployment rather than depending on teleoperated or planner-generated references. Both the planner and the tracking policy are otherwise standard; beyond demonstrating this pairing, we analyze why the decomposition is effective, showing that the low-level policy shrinks the search space and stabilizes rollouts while keeping high-level optimization online retains a test-time flexibility that offline-trained high-level policies sacrifice. Closest to our setting, Kim et al. [22] combine a sampling-based planner with a low-level RL policy for high-speed quadruped locomotion and navigation on complex, discrete terrain. Sumo shares this structure but studies dynamic, contact-rich *loco-manipulation*, where the planner must reason through the pretrained policy about object dynamics and manipulation objectives rather than terrain traversal. Planning over pretrained robot policies has also been explored in tabletop manipulation [33, 17] where the base policy is trained via imitation learning.

Finally, an alternative high-level policy is a *learned* decision maker such as a vision-language-action (VLA) model [23, 4]. VLAs offer semantic, open-world generalization from large-scale demonstrations but currently operate at low rates on quasi-static, gripper-centric tasks, and new embodiments or dynamic skills require substantial data collection; Sumo instead requires an object model and a cost function, and in return provides high-rate replanning for dynamic whole-body contact. The two are complementary: a VLA could supply goals, object models, or cost functions that Sumo executes through dynamic replanning (Sec. VI).

III. SUMO

In this section, we provide a detailed description of our hierarchical control framework, which we call Sumo, that uses a high-level sample-based controller to steer a pre-trained whole-body control policy to perform whole-body loco-manipulation with quadruped and humanoid robots.

A. Overview

Our system, detailed in Figure 2, uses a pre-trained steerable whole-body control policy that takes in the current state and desired torso, arm, and optionally leg commands of the robot and outputs the low-level joint commands for the quadruped or humanoid robot at 50Hz. At the high-level, we use a sample-based MPC policy (e.g., MPPI, CEM, etc.) that takes in the current state estimate and updates the desired torso and arm commands for the low-level locomotion policy at 20Hz. Together, our framework enables complex loco-manipulation skills that can only be achieved through whole-body contact-rich coordination, such as uprighting and stacking a fallen tire heavier than the robot's lifting capacity (Fig. 7 (a) (e)), and uprighting and dragging a crowd control barrier larger than the robot itself (Fig. 7 (b) (f)).

TABLE I: Timing comparison of MuJoCo simulation of 32 parallel rollouts over 1.5 seconds with and without the low-level policy on an Intel Core i7-12700K CPU.

Method	Rollout Time (ms)
MuJoCo with low-level policy	43.45 ± 1.88 ms
MuJoCo only	21.73 ± 1.86 ms

B. Pre-Trained Whole-Body Control Policy

For the low-level policy, we use an existing whole-body control policy trained via RL that takes high-level commands and outputs the joint-level commands for the entire robot. Methods for training humanoid and quadruped policies have been studied extensively in the literature and are widely available [15, 8]. We do not aim to improve whole-body control capabilities in this paper, and instead focus on their downstream applications for loco-manipulation. Specifically, we use the Relic policy [55] which is designed for multi-limb loco-manipulation on the Spot quadruped robot. In particular, the Relic policy enables stable gaits with only three legs and allows the robot to use the fourth leg as a manipulator in addition to its arm and torso. This setup allows the high-level sampling-based controller to automatically reason about using multiple limbs or a combination of limbs and torso to manipulate an otherwise challenging object.

For the G1 humanoid robot, we use the standard velocity-tracking policy from MJLab [32] that takes in the desired torso velocity commands. While this policy is not explicitly trained for loco-manipulation and does not take in the desired arm commands, we find that simply overriding the arm commands with target commands from the high-level sampling-based MPC is also effective.

While we believe these are natural choices for the Spot and G1 robots, one can similarly choose other policies over arbitrary input spaces from which the sample-based controller can sample; we return to interface design considerations below.

C. Policy-in-the-Loop Parallel Rollouts

Let $\mathbf{s}$ denote the state of the full system, stacking the robot's floating-base pose, joint positions, and their velocities together with the pose and velocity of the manipulated object, and let $\mathbf{u}$ denote the joint-level control applied to the robot. A standard multi-body simulator advances this state as

$$\mathbf{s}_{k+1} = f(\mathbf{s}_k, \mathbf{u}_k). \tag{1}$$

Algorithm 1: Sumo Planning Iteration

Input: state estimate $\mathbf{s}_0$; previous plan spline $\mathcal{S}$; per-node noise scales $\boldsymbol{\sigma}$; task cost J

$\bar{\mathbf{A}} \leftarrow \mathcal{S}(t)$; // warm start: time-shift previous plan

for *each rollout* i **do** // in parallel

  $\mathbf{A}^i \leftarrow \text{clip}\big(\bar{\mathbf{A}} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}^i\big), \quad \boldsymbol{\epsilon}^i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;

  $\mathbf{a}^i(t) \leftarrow$ interpolate control points $\mathbf{A}^i$;

  $\mathbf{s}^i_{k+1} = f'\big(\mathbf{s}^i_k, \mathbf{a}^i(t_k)\big)$, $\mathbf{s}^i_0 = \mathbf{s}_0$; // policy-in-the-loop rollout

  $J^i \leftarrow \sum_k J\big(\mathbf{s}^i_k, \mathbf{a}^i(t_k)\big)$;

retain the unperturbed plan $\bar{\mathbf{A}}$ as one rollout;

$(\bar{\mathbf{A}}, \boldsymbol{\sigma}) \leftarrow \textsc{Update}\big(\{\mathbf{A}^i, J^i\}\big)$; // optimizer-specific distribution update

$\mathcal{S} \leftarrow$ spline through the updated control points $\bar{\mathbf{A}}$; execute until the next iteration;

Previous sample-based MPC searches directly over sequences of $\mathbf{u}$ [1, 25], Fig. 3 (a). Our low-level policy π instead maps the state and a command $\mathbf{c}$ to joint-level controls, $\mathbf{u} = \pi(\mathbf{s}, \mathbf{c})$, while the high-level planner samples actions $\mathbf{a}$ that are mapped to policy commands, $\mathbf{c} = g(\mathbf{a})$, as detailed in Sec. III-D. A key component of our system is therefore a physics rollout that composes the policy with the simulator, giving the policy-in-the-loop dynamics

$$\mathbf{s}_{k+1} = f'(\mathbf{s}_k, \mathbf{a}_k) \triangleq f\big(\mathbf{s}_k,\, \pi(\mathbf{s}_k, g(\mathbf{a}_k))\big), \tag{2}$$

so that the planner searches over sequences of $\mathbf{a}$ rather than $\mathbf{u}$, Fig. 3 (b). In practice, π is evaluated at its 50Hz control rate and the resulting $\mathbf{u}$ is held constant across the intervening simulator steps. This design choice is critical for online sample efficiency for several reasons: First, this allows the sample-based MPC to sample in a lower-dimensional action space, mitigating the curse of dimensionality commonly seen for search algorithms. Second, with the low-level locomotion policy as a stabilizing controller, we avoid the well-known divergence issues associated with single-shooting methods applied to unstable dynamics that commonly occur with legged robots. Finally, because the whole-body control policy handles the locomotion part of the problem, this setup allows us to design much simpler cost function that only needs to encode manipulation objectives.

We implement the policy-in-the-loop parallel simulation by augmenting the CPU-based MuJoCo physics engine [44] in C++ using thread pool. In Table I we report the rollout times of 32 parallel rollouts over 1.5 seconds with and without the low-level policy on an Intel Core i7-12700K CPU. Indeed, the low-level policy adds overhead to the physics simulation, but the total times are faster than the 20Hz or 50ms update rate of the sample-based MPC. A natural question is whether the rollouts should run on a CPU or GPU. In general, CPU parallelization offers low latency at lower throughput, whereas GPU batching offers high throughput at the cost of higher latency. In real-time control we prefer low latency, with faster replanning often preferred over higher-reward solutions computed at slower rates [1]. GPU simulators [11] remain an interesting direction for scaling test-time search when hundreds to thousands of samples are necessary. The policy-inference overhead in Table I remains modest because RL amortizes optimization at training time, which is what makes policy-in-the-loop rollouts practical compared to embedding a QP-based policy in every rollout.

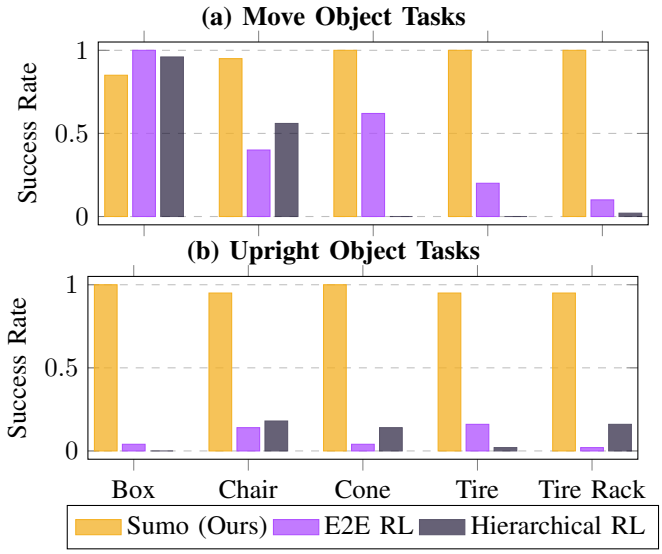

Fig. 4: Top: comparison of Sumo (yellow, ours), E2E RL (purple), and hierarchical RL (navy, HRL) on pushing five different objects to a goal. Sumo generalizes to new objects by replacing the object model at test time, whereas E2E RL and HRL policies trained only on box pushing fail on the other objects. Bottom: Sumo generalizes to uprighting objectives by changing the planner cost at test time, whereas the same E2E RL and HRL policies fail without additional training.

D. High-Level Sample-Based MPC

Naturally, the high-level sample-based MPC can sample over any action space supported as inputs to the low-level policy. In practice, we choose to sample actions over a compact, task-relevant control vector, then rely on the low-level policy to map this to a full whole-body command. Here, we describe the details for the Spot robot with the Relic policy, but similar principles apply to other robots and low-level policies. Concretely, we specify the action space of $\mathbf{a}$ and the map g from planner actions to policy commands $\mathbf{c}$ introduced in Sec. III-C.

For the Spot robot, the full $\mathbf{c}_{\text{policy}} \in \mathbb{R}^{25}$ includes the $\text{SE}(2)$ velocities for the base $\mathbf{c}_{\text{base}} \in \mathbb{R}^3$, the arm joint-angle targets $\mathbf{c}_{\text{arm}} \in \mathbb{R}^6$, gripper position $\mathbf{c}_{\text{gripper}} \in \mathbb{R}^1$, leg joint angle targets $\mathbf{c}_{\text{leg}} \in \mathbb{R}^3$ for all four legs, and torso pitch, roll, and height targets $\mathbf{c}_{\text{torso}} \in \mathbb{R}^3$. By default, we only sample over the base velocities $\mathbf{a}_{\text{base}} \in \mathbb{R}^3$ and arm joint angle targets $\mathbf{a}_{\text{arm}} \in \mathbb{R}^6$ for the arm, excluding the gripper. To pad remaining policy commands, we set leg targets to zero (the low-level policy automatically outputs leg joint commands to track desired base velocities) and use default values for the torso pitch, roll, and height targets and set the gripper to a closed position. This base setup is sufficient for many loco-manipulation tasks but can be further enhanced by reasoning over legs, torso, and

gripper commands at test time for tasks that require multi-limb coordination or gripper dexterity.

To reason over the torso roll, pitch, and height commands, we can simply add those dimensions $\mathbf{a}_{\text{torso}} \in \mathbb{R}^3$ to the action space. To enable the robot to use front legs for manipulation, we can additionally sample over $\mathbf{a}_{\text{leg}} \in \mathbb{R}^7 = [\mathbf{s}_{\text{leg}}, \mathbf{c}_{\text{leg}}]$ where $\mathbf{s}_{\text{leg}} \in [-1, 1]$ is a selection variable and $\mathbf{c}_{\text{leg}} \in \mathbb{R}^6$ is the leg joint angle targets, then use the selection variable to mask the leg joint angle targets:

$$\mathbf{c}_{\text{leg}}^{\text{masked}} = \begin{cases} [\mathbf{c}_{\text{leg}}[0{:}3], 0, 0, 0] & \text{if } s_{\text{leg}} < -0.5 \text{ (FL)} \\ [0, 0, 0, \mathbf{c}_{\text{leg}}[3{:}6]] & \text{if } s_{\text{leg}} > 0.5 \text{ (FR)} \\ [0, 0, 0, 0, 0, 0] & \text{else (no legs)} \end{cases} \tag{3}$$

Note that it is also possible to use rear legs for manipulation, but our implementation chooses to only focus on front legs as they cover most practical loco-manipulation tasks. Similarly, we can control the gripper via a binary action variable $\mathbf{a}_{\text{gripper}} \in [-1, 1]$ that maps to open and close positions:

$$\mathbf{c}_{\text{gripper}} = \begin{cases} \mathbf{c}_{\text{gripper close}} & \text{if } \mathbf{a}_{\text{gripper}} > 0 \\ \mathbf{c}_{\text{gripper open}} & \text{if } \mathbf{a}_{\text{gripper}} \leq 0 \end{cases} \tag{4}$$

This design allows us to flexibly choose task-specific action spaces to be any combination of $\mathbf{a} = [\mathbf{a}_{\text{base}}, \mathbf{a}_{\text{arm}}, \mathbf{a}_{\text{torso}}, \mathbf{a}_{\text{leg}}, \mathbf{a}_{\text{gripper}}]$ for the sample-based MPC. Additionally, the selection variables allow the controller to reason over different manipulation modalities at test time. Note that our sample-based MPC naturally reasons over both continuous and discrete decision variables which would be challenging for traditional gradient-based optimal control algorithms.

For the G1 humanoid robot, we similarly sample over the $\mathrm{SE}(2)$ velocities for the torso and the arm joint angle targets for the arm. While it is straight forward to enable more inter-limb coordination by using a more steerable underlying low-level policy [28], we leave this investigation as a future direction.

In our experience, the low-level policy interface properties that matter most for planning are 1) a low-dimensional command space, 2) consistent closed-loop command effects (so sampled perturbations produce predictable behavior variations), and 3) stability under rapidly changing commands. A policy conditioned on per-limb end-effector targets is an appealing alternative: it would give the planner direct authority over contact locations and further shrink the search space, at the cost of being harder to train with high tracking fidelity near contact, where forces rather than positions dominate. As more steerable whole-body trackers emerge [28, 27], we expect the planner-over-policy recipe to benefit directly.

E. Planner Implementation Details

Algorithm 1 summarizes one planning iteration. Our sampling distribution is a Gaussian with a *diagonal* covariance: noise is sampled independently per spline control point and action dimension, with no cross coupling. Actions are normalized—sampled in normalized units, clipped to the low-level policy's command limits, and affinely mapped to physical commands. The discrete variables ($\mathbf{s}_{\text{leg}}$, $\mathbf{a}_{\text{gripper}}$) are treated as continuous values and only thresholded via (3) and (4). The planner is warm-started by time-shifting the previous plan to seed the distribution mean, and the unperturbed shifted plan is retained as one of the rollouts. Finally, commands are linearly interpolated between control points by default with options for zeroth-order or cubic interpolations. The sampling noise follows the linear schedule of Sec. IV: the beginning of the plan is executed before the next replan and has been refined by previous iterations, so it benefits from low noise, while the tail of the horizon is least optimized and benefits from wider exploration.

The distribution update in Algorithm 1 produces a new mean $\bar{\mathbf{A}}$ and noise scale $\boldsymbol{\sigma}$ from the sampled control sequences and their costs. The per-node noise schedule (Sec. IV) shapes $\boldsymbol{\sigma}$ across the horizon at sampling time. Note that sample-based planners (CEM, MPPI, CMA-ES, etc.) are related zeroth-order policy-improvement updates and have been analyzed under a common framework in recent work [50, 37], so the specific rule is not central to our framework. We adopt CEM for its simplicity, and our open-source implementation exposes other planners as drop-in alternatives.

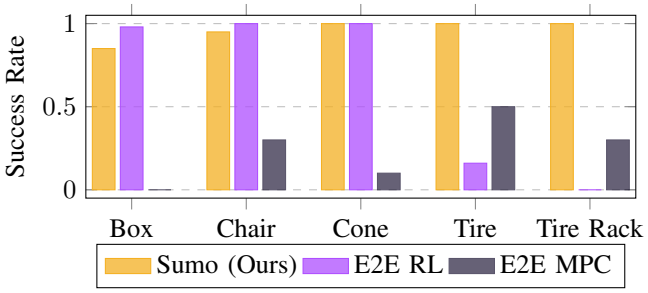

Fig. 5: Comparing Sumo (ours, yellow) to end-to-end RL (purple) and MPC (navy) on five loco-manipulation tasks that ask the robot to move an object to a goal. Sumo achieves high success across all objects. End-to-end RL is competitive on the box, chair, and cone but degrades sharply on the tire and tire rack, while end-to-end MPC struggles across the board.

IV. Experimental Analysis

In this section, we use simulation experiments in MuJoCo [44] to analyze three advantages of Sumo: hierarchical structure simplifies loco-manipulation, test-time search enables generalization, and Sumo achieves higher sample efficiency than a hierarchical RL policy.

A. Task Definition and Experiment Setup

In this section, we consider two types of loco-manipulation tasks on the Spot quadruped robot: moving an object to a goal position (Move) and reorienting an object to the upright orientation (Upright). In both cases, we consider five objects covering a range of size, weight, and geometry representative of real-world scenarios: a 1.5 kg box, a chair, a car tire, a tire rack, and a traffic cone.

For Move tasks, we consider success if the object is within 0.1 meters of the goal and velocity is less than 0.05 m/s within 30 seconds. For Upright tasks, we consider task success if

the object is within 0.1 radians of the upright orientation and angular velocity is less than 0.05 rad/s within 30 seconds.

To simplify the analysis, we restrict the Sumo high-level planner to only control the torso $\mathrm{SE}(2)$ velocities and joint angle targets for the arm in this section and leave exploring more expressive policy behaviors to Sec. V. Additionally, we design a minimal cost function for Move tasks:

$$J_{\text{Move}} = w_{\text{goal}} \cdot \|\mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{goal}}\|_2 + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{obj}}\|_2 + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2 \quad (5)$$

where $\mathbf{p}_{\text{obj}}$, $\mathbf{p}_{\text{goal}}$, and $\mathbf{p}_{\text{gripper}}$ are the positions of the object, goal, and gripper in world frame, respectively, $\mathbf{v}_{\text{obj}}$ is the linear velocity of the object, and $w_{\text{goal}}, w_{\text{gripper}}, w_{\text{vel}}$ are weighting coefficients. Similarly, for Upright tasks:

$$J_{\text{Upright}} = w_{\text{Upright}} \cdot (1 - \hat{\mathbf{z}}_{\text{obj}} \cdot \hat{\mathbf{z}}_{\text{world}}) + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{obj}}\|_2 \quad (6)$$

where $\hat{\mathbf{z}}_{\text{obj}}$ is the object body axis that points upward in the target upright pose, expressed in world frame, and $\hat{\mathbf{z}}_{\text{world}} = [0, 0, 1]^\top$. This axis-alignment cost vanishes exactly at the upright orientation and avoids the ambiguities of comparing quaternions directly.

For all Sumo experiments, we use a Cross-Entropy Method (CEM) optimizer with a prediction horizon of 1.5 seconds, 32 rollouts, and update the action distribution with three elite samples at a frequency of 20Hz. We sample over four spline control points along the prediction horizon and interpolate the remaining actions. Finally, we linearly schedule the noise variance ramping from 0.02 to 0.6 over the prediction horizon. Implementation details of the optimizer are provided in Sec. III-E. The cost weights are shared across all five objects without per-object tuning: w_{goal}=60, w_{gripper}=4, w_{vel}=20 for Move, and w_{Upright}=100, w_{gripper}=0.5 for Upright. All experiments are conducted synchronously on a desktop computer equipped with an Intel Core i7-12700K CPU and 64GB of RAM. Current robot and object states are provided by the ground-truth simulator.

B. Hierarchical Structure Simplifies Loco-Manipulation

We first evaluate whether hierarchical structure simplifies loco-manipulation. Fig. 5 compares Sumo against both real-time MPC (E2E MPC) and end-to-end RL (E2E RL) on the Move tasks. This comparison isolates the benefit of planning in the command space of a pre-trained whole-body controller rather than optimizing directly over joint-level actions or learning a task-specific end-to-end policy.

For the E2E MPC baseline, we use the same CEM optimizer, which is used in Sumo and common for locomotion and manipulation [25, 14, 1], whose action space is the joint-level commands of the robot. We design rewards to include terms for both locomotion and manipulation objectives:

$$J_{\text{E2E MPC}} = J_{\text{Locomotion}} + J_{\text{Move}} \quad (7)$$

where $J_{\text{Locomotion}}$ is the locomotion-related cost from [14] and J_{Move} is defined in (5). Additionally, we allow the E2E MPC to update at 50Hz to match Sumo's low-level policy frequency.

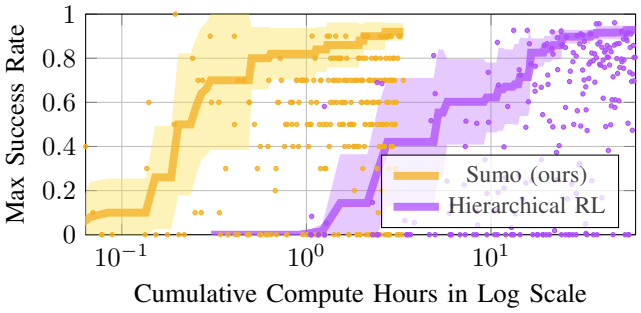

Fig. 6: Maximum success rate vs compute time comparison between Sumo (blue) and hierarchical RL (red) baseline during hyperparameter Bayesian optimization. Solid lines represent the mean across 5 optimization runs, and the shaded region shows ± 1 standard deviation. Individual success rates are shown as scatter points. Our method achieves similar asymptotic performance compared to hierarchical RL in an order of magnitude less wall-clock time.

For the E2E RL baseline, we train a state-based, goal-conditioned single-task policy that controls the 19-DoF joints of Spot to move an object to a goal location. We train the policy using PPO in MJLab [32] with 4096 parallel environments for 5000 iterations. We use 15 reward terms, including terms that encourage natural walking through a phase-based gait and foot-height schedule, progress toward the manipulation goal, and shaping that encourages the gripper to approach the object and the robot to stay behind it. The policy also receives bonuses when it reaches the goal and keeps the object within 0.2m, which we count as success. As with Sumo, we tune the reward until the policy reliably solves the box task, then apply the same setup to four additional objects without further task-specific reward engineering. In both baselines, we make a best-faith effort to tune hyperparameters to achieve the best performance.

We find that Sumo successfully solves all five Move tasks with at least 80% success across 20 evaluation trials, with some tasks at 100% success. In contrast, E2E MPC struggles to consistently solve these tasks, achieving at most 50% success. E2E RL is competitive on the box, chair, and cone, but degrades sharply on more complex geometries such as the tire and tire rack. We summarize these results in Fig. 5.

These experiments show that hierarchical structure simplifies loco-manipulation in two complementary ways. Compared to E2E MPC, planning in the task space of a pre-trained low-level policy reduces the effective search space and stabilizes the robot's contact-rich dynamics. Compared to E2E RL, Sumo achieves similar or better success with only 3 reward terms and no task-specific retraining or tuning, whereas E2E RL requires 15 reward terms and about 2 hours of GPU compute per task. This comparison also motivates the next question: once a low-level whole-body controller is fixed, should the high-level policy be learned or optimized online at test time?

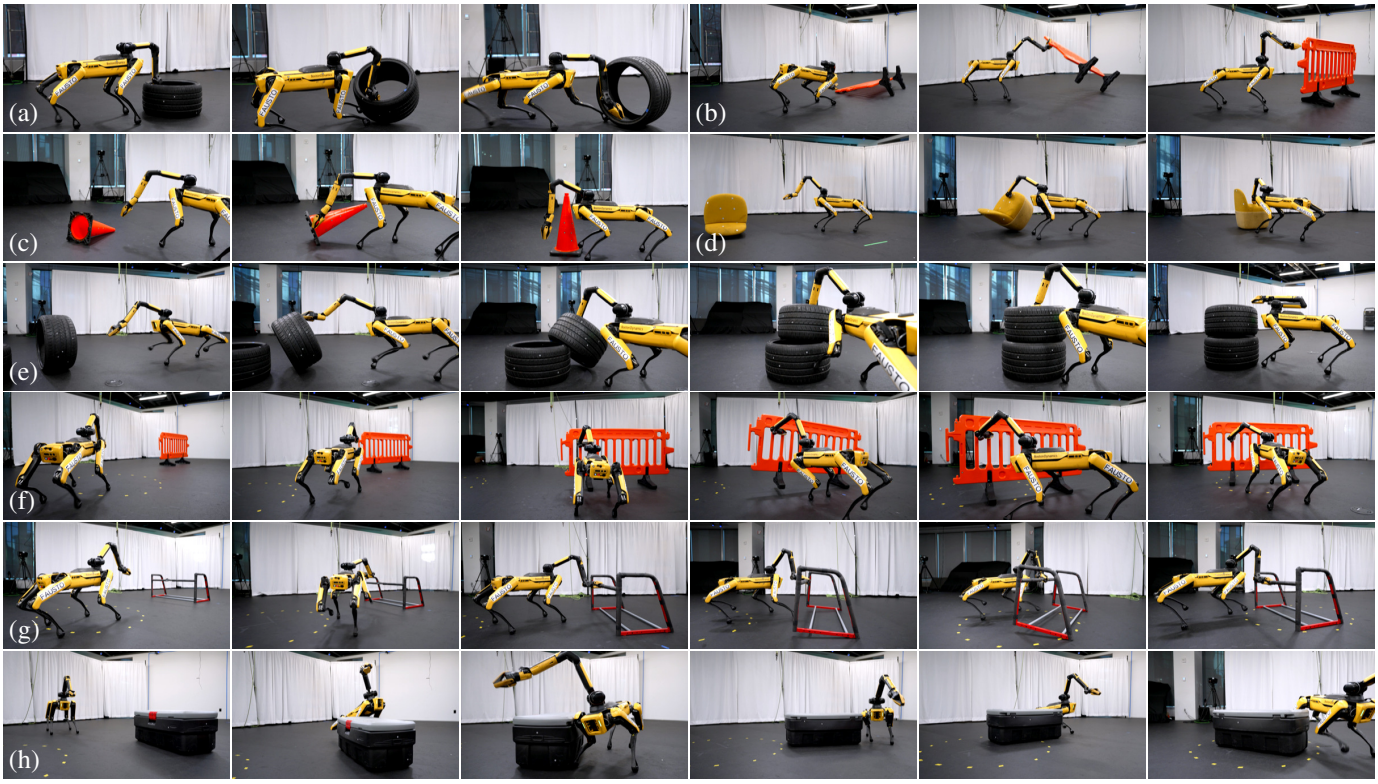

Fig. 7: Freeze-frame sequences showing Spot robot task progressions: (a) uprighting a tire, (b) uprighting a crowd control barrier, (c) uprighting a traffic cone, (d) uprighting a chair, (e) stacking a two tires, (f) dragging a crowd control barrier, (g) dragging a tire rack, and (h) pushing a heavy box.

C. Test-Time Search Enables Generalization

We next ask whether the high-level decision-making module should be learned or optimized online. Our experiments show that keeping this layer online is what enables generalization in our setting. Here, generalization means reusing the same learned controller on new objects and new task objectives by changing only the planner's object model or cost at test time, without retraining. This distinction matters because RL policies are typically trained for a *known* distribution of environments and objects, whereas open-world manipulation introduces geometries, masses, and contact conditions that are difficult to model comprehensively at training time.

Fig. 4 (a) shows that a hierarchical RL policy trained to move a 1.5 kg box to a goal does not generalize reliably once object geometry and weight move beyond its training distribution, even after randomizing the box size, weight, and friction. While the hierarchical RL policy achieves a 100% success rate on this training task, its performance degrades quickly on objects such as the tire and traffic cone. In contrast, our method can generalize to different objects *without additional training* by simply replacing the object model. All tasks here use the same cost function and hyperparameters described in Eq. (5).

The hierarchical RL policy commands the *same* pretrained low-level policy as Sumo through the same command space, and observes the robot state, object pose and velocity, and goal. Neither high-level policy is conditioned on explicit object parameters such as geometry or inertia. Object information enters Sumo only through the planner's simulation model and has no analogue in the RL training setting.

Fig. 4 (b) shows the same advantage for different task objectives. By changing the planner cost function at test time, similar in spirit to guidance for diffusion models [18], we replace the goal-directed objective with the orientation objective in Eq. (6) and enable the robot to reliably upright different objects without additional training. In contrast, the same RL policy cannot be steered to upright different objects without additional training.

D. Sample-Based MPC Provides High Practical Efficiency

Finally, we compare the practical iteration speed of sample-based MPC and RL during hyperparameter tuning. The goal of this experiment is not a hardware-normalized compute comparison, but rather a wall-clock comparison of the tuning loop practitioners would run in practice. In Fig. 6, we mimic this process by performing a Bayesian optimization over the cost weights in (5) for the Move Box task for both Hierarchical RL and Sumo to find the weights that maximize the success rate. We perform five independent optimization runs and track the maximum success rate over time across 50 trials. Both methods achieve similar asymptotic performance, but Sumo reaches the same performance with an order-of-magnitude less compute time. For Sumo, we measure the compute time as CPU hours on a desktop computer equipped with an Intel Core i7-12700K CPU and 64GB of RAM, and RL training time is measured as GPU hours on an NVIDIA RTX A6000 GPU.

TABLE II: Real-world completion time and success rate performance of Spot loco-manipulation tasks across 10 trials.

Task Name	Completion Time	Successes	Time Limit
Tire Upright	9.2 ± 4.7s	10/10	30s
Barrier Upright	10.5 ± 7.1s	9/10	30s
Cone Upright	10.2 ± 7.9s	9/10	30s
Chair Upright	27.3 ± 19.1s	8/10	60s
Tire Stack	16.5 ± 8.4s	8/10	30s
Barrier Drag	20.2 ± 6.7s	9/10	30s
Tire Rack Drag	19.1 ± 6.2s	9/10	30s
Rugged Box Push	38.3 ± 16.9s	10/10	90s

V. Autonomous Demonstrations with Sumo

In this section, we move from controlled simulation analysis to full-task demonstrations on eight Spot tasks in the real world and four G1 tasks in simulation. Together, these case studies test whether the same hierarchical framework can handle diverse manipulation modes and object interactions beyond the simplified analysis tasks. Unlike the simplified cost functions used in Sec. IV, these demonstrations use task-specific cost functions tailored to each task; full descriptions are provided in the appendix. The additional terms address what the idealized analysis setting does not exercise: safety terms penalize plans that risk falls, grasp-detection terms let the planner verify that the gripper actually holds the object, and approach/standoff terms shape how the robot engages the object, biasing the search away from local minima that are harmless in simulation but costly on hardware.

A. Spot Quadruped Loco-Manipulation

In the first case study, we evaluate Sumo on a diverse set of challenging loco-manipulation tasks on the Boston Dynamics Spot quadruped. These tasks stress three recurring difficulties: objects that are larger than the robot or heavier than the arm payload, contact conditions and geometries that create large sim-to-real gaps, and distinct manipulation modes such as uprighting, stacking, dragging, and pushing. The tasks and corresponding performance are summarized in Table II.

For all hardware deployments, we use the same set of optimizer parameters as detailed in Sec. IV. To estimate the robot and object states, we fuse the Spot robot joint encoder reading at 333Hz with motion-capture (MoCap) measurements for torso and object positions and orientations at 120Hz using a low-pass filter. We compute the dynamics rollouts and CEM updates asynchronously on a desktop computer equipped with an AMD Threadripper Pro 5995WX CPU with 64 cores, which sends joint-level commands to the Spot robot via WiFi. State estimation thus relies on external MoCap and planning runs off-board: these experiments demonstrate the control approach rather than full onboard autonomy (see Sec. VI).

Below, we briefly describe each task to provide qualitative context for the quantitative results in Table II.

Tire Upright: In Fig. 7 (a), the robot is tasked to upright a tire of 15kg, which is heavier than the maximum lifting capacity of 11kg of the Spot arm. Additionally, the rubber tire presents hard-to-model geometries and friction properties, causing a large sim-to-real gap. The robot solves the task using a combination of its arm, torso, and legs.

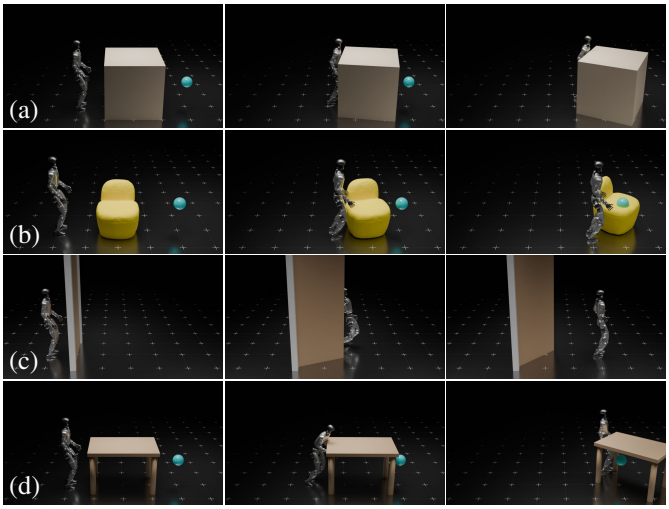

Fig. 8: Freeze-frame sequences showing G1 humanoid task progression: (a) pushing a box, (b) pushing a chair, (c) opening a door, and (d) pushing a table to a goal position (blue sphere).

Crowd Barrier Upright: In Fig. 7 (b), the robot is tasked to upright a crowd barrier of 16kg from a lying position using its arm and gripper.

Traffic Cone Upright: In Fig. 7 (c), the robot is tasked to upright a traffic cone of 3.5kg using its arm.

Chair Upright: In Fig. 7 (d), the robot is tasked to upright a chair of 16.5kg using its arm and body.

Tire Stack: In Fig. 7 (e), the robot is tasked to stack a 15kg tire on top of another tire. In addition to large weight and deformation, the friction coefficient between the rubber tires is high, which current simulators struggle to simulate accurately, causing an even larger sim-to-real gap. The robot solves the task using a combination of its arm, torso, and legs.

Crowd Barrier Drag: In Fig. 7 (f), the robot is tasked to drag a crowd barrier of 15kg in upright orientation. The robot uses the gripper to grasp the barrier and drags it towards a goal position with its arm and body.

Tire Rack Drag: In Fig. 7 (g), the robot is tasked to drag a tire rack of 10kg. The robot uses the gripper to grasp the tire rack and drags it towards a goal position.

Rugged Box Push: In Fig. 7 (h), the robot is tasked to push a 20kg box to a goal using its arm and body.

B. G1 Humanoid Loco-Manipulation

In the second case study, we show that Sumo can also be applied to humanoid robots. Through simulated experiments using a Unitree G1, our method uses a pre-trained locomotion policy that is widely available [32], and solves loco-manipulation tasks such as pushing a box and opening a door. Simulation performances are summarized in Table III.

Box Pushing In Fig. 8 (a), the robot is tasked to push a 10kg box to a target position using its arms and body.

Chair Pushing In Fig. 8 (b), the robot is tasked with pushing a 16.5kg chair to a goal using its arms and body.

Door Opening In Fig. 8 (c), the robot is asked to open a door using its arm and body.

TABLE III: Performance of G1 humanoid loco-manipulation tasks in simulation.

Task Name	Completion Time (s)	Successes	Time Limit
G1 Box Push	11.83 ± 2.97 s	9/10	30s
G1 Door Open	4.73 ± 0.98 s	10/10	30s
G1 Chair Push	6.86 ± 0.288 s	10/10	30s
G1 Table Push	4.86 ± 1.65 s	8/10	30s

Table Pushing In Fig. 8 (d), the robot is tasked with pushing a 10kg table to a goal using its arms and body.

C. Open-Source Benchmark

We release the tasks in this paper as an open-source benchmark for whole-body loco-manipulation. The release contains simulation environments for the analysis tasks of Sec. IV and the demonstration tasks of this section on both robots; MJCF models of the robots and all manipulated objects, with masses and geometries matched to our physical objects; pretrained low-level policies integrated into the policy-in-the-loop C++ rollout backends of Sec. III; complete cost implementations with all weights, success criteria, and reset distributions; CEM, MPPI, and predictive sampling as interchangeable optimizers, with an interactive visualizer; and utilities to record and replay rollout trajectories. Tasks share a common interface, so new objects and objectives require only an object model and a cost function. The code is publicly available at https://github.com/rai-opensource/sumo, and videos of all tasks are available on the project page at https://sumo.rai-inst.com.

VI. Conclusions and Future Work

This paper explores the combination of RL pre-training and test-time sample-based MPC for whole-body loco-manipulation. We find this approach, which we call Sumo, excels at challenging tasks involving large and heavy objects that cannot be solved via pick-and-place-style manipulation. We also find that the decomposition of difficult loco-manipulation tasks into a two-stage optimization problem makes the sub-problems much more tractable. Additionally, we show that using sample-based MPC as a high-level policy enables test-time task flexibility and generalization that is difficult for RL.

Sumo still has several shortcomings and avenues for future work: First, our current system depends on off-board estimation and planning. Recent powerful compact-form-factor computers, such as the M-series Mac mini, make deploying Sumo-style algorithms computationally tractable. However, we leave fully on-board deployment on the Spot and G1 robots for future work. Second, as an algorithm relying entirely on sim-to-real transfer, the "sim-to-real gap" remains a significant challenge. While modeling the robot and object dynamics perfectly is impossible and often not necessary, building a "good enough" model for control remains an art that requires some trial and error. In fact, we believe real-time MPC as the high-level policy significantly accelerates this tuning loop compared to RL training. Nevertheless, Sumo can undoubtedly benefit from system-identification and model-learning methods that update object parameters on the fly from real-world interactions. Finally, Sumo, in its current form, does not leverage any human priors. Future work should explore incorporating pre-trained foundation models that can automatically guide robot behaviors without manually engineered rewards—for example, foundation and agentic models that supply goals, object models, or cost functions, which Sumo then executes through dynamic whole-body planning and control.

Acknowledgments

We thank Emmanuel Panov for supporting hardware deployment and Jonathan Foster for building 3D object meshes used in the experiments. We also thank Swaminathan Gurumurthy for providing thoughtful feedback and discussions. Finally, we thank Yuval Tassa and Taylor Howell for answering MuJoCo-related questions. This work was done in part during an internship at the RAI Institute.

Coding agents like Claude Code and OpenAI Codex were used to assist in generating parts of the code for experiments included in the paper. AI systems were also used to proofread and correct grammatical errors in the manuscript.

References

[1] Juan Alvarez-Padilla, John Z Zhang, Sofia Kwok, John M Dolan, and Zachary Manchester. Real-time whole-body control of legged robots with model-predictive path integral control. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14721–14727. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11128271.

[2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, volume 30, pages 5048–5058, 2017.

[3] Philip Arm, Mayank Mittal, Hendrik Kolvenbach, and Marco Hutter. Pedipulate: Enabling manipulation skills using a quadruped robot's leg. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5717–5723, 2024. doi: 10.1109/ICRA57147.2024.10611307.

[4] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Gerardo Bledt, Matthew J. Powell, Benjamin Katz, Jared Di Carlo, Patrick M. Wensing, and Sangbae Kim. Mit cheetah 3: Design and control of a robust, dynamic quadruped robot. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2245–2252, 2018. doi: 10.1109/IROS.2018.8593885.

[6] Jin Cheng, Dongho Kang, Gabriele Fadini, Guanya Shi, and Stelian Coros. Rambo: Rl-augmented model-based whole-body control for loco-manipulation. *IEEE Robotics and Automation Letters*, 10(9):9462–9469, 2025. doi: 10.1109/LRA.2025.3594984.

[7] Xuxin Cheng, Ashish Kumar, and Deepak Pathak. Legs as manipulator: Pushing quadrupedal agility beyond locomotion. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[8] Xuxin Cheng, Kexin Shi, Ananye Agarwal, and Deepak Pathak. Extreme parkour with legged robots. *IEEE International Conference on Robotics and Automation (ICRA)*, pages 11443–11450, 2024. doi: 10.1109/ICRA57147.2024.10610200.

[9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.

[10] Simon Le Cleac'h, Taylor Howell, Shuo Yang, Chi-Yen Lee, John Zhang, Arun Bishop, Mac Schwager, and Zachary Manchester. Fast Contact-Implicit Model-Predictive Control. *IEEE Transactions on Robotics and Automation*, January 2024. doi: 10.48550/arXiv.2107.05616.

[11] Google DeepMind and NVIDIA. Mujoco warp: Gpu-optimized version of the mujoco physics simulator. https://github.com/google-deepmind/mujoco_warp, 2025. Accessed: 2025-11-19.

[12] Ruben Grandia, Fabian Jenelten, Shaohui Yang, Farbod Farshidian, and Marco Hutter. Perceptive locomotion through nonlinear model-predictive control. *IEEE Transactions on Robotics*, 39(5):3402–3421, 2023. doi: 10.1109/TRO.2023.3275384.

[13] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

[14] Taylor Howell, Nimrod Gileadi, Saran Tunyasuvunakool, Kevin Zakka, Tom Erez, and Yuval Tassa. Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo. *arXiv preprint arXiv:2212.00541*, 2022.

[15] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario C. Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019. doi: 10.1126/scirobotics.aau5872.

[16] D. H. Jacobson and D. Q. Mayne. *Differential Dynamic Programming*. Elsevier, 1970.

[17] Arnav Kumar Jain, Vibhakar Mohta, Subin Kim, Atiksh Bhardwaj, Juntao Ren, Yunhai Feng, Sanjiban Choudhury, and Gokul Swamy. A smooth sea never made a skilled sailor: Robust imitation via learning to search. *arXiv preprint arXiv:2506.05294*, 2025. URL https://arxiv.org/pdf/2506.05294.

[18] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022.

[19] Fabian Jenelten, Junzhe He, Farbod Farshidian, and Marco Hutter. Dtc: Deep tracking control. *Science Robotics*, 9(86):eadh5401, 2024. doi: 10.1126/scirobotics.adh5401.

[20] Se Hwan Jeon, Ho Jae Lee, Seungwoo Hong, and Sangbae Kim. Residual mpc: Blending reinforcement learning with gpu-parallelized model predictive control. *arXiv preprint arXiv:2510.12717*, 2025. URL https://arxiv.org/abs/2510.12717.

[21] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017. doi: 10.1137/16M1062569.

[22] Hyeongjun Kim, Hyunsik Oh, Jeongsoo Park, Yunho Kim, Donghoon Youm, Moonkyu Jung, Minho Lee, and Jemin Hwangbo. High-speed control and navigation for quadrupedal robots on complex and discrete terrain. *Science Robotics*, 10(102):eads6192, 2025.

[23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[24] Albert H. Li, Preston Culbertson, Vince Kurtz, and Aaron D. Ames. Drop: Dexterous reorientation via online planning. *arXiv preprint arXiv:2409.14562*, 2024. URL https://arxiv.org/abs/2409.14562.

[25] Albert H. Li, Brandon Hung, Aaron D. Ames, Jiuguang Wang, Simon Le Cleac'h, and Preston Culbertson. Judo: A user-friendly open-source package for sampling-based model predictive control. In *Proceedings of the Workshop on Fast Motion Planning and Control in the Era of Parallelism at Robotics: Science and Systems (RSS)*, 2025. URL https://github.com/bdaiinstitute/judo.

[26] Weiwei Li and Emanuel Todorov. Iterative linear quadratic regulator design for nonlinear biological movement systems. In *Proceedings of the First International Conference on Informatics in Control, Automation and Robotics (ICINCO)*, volume 1, pages 222–229. INSTICC, SciTePress, 2004. doi: 10.5220/0001143902220229.

[27] Yuhan Li, Peiyuan Zhi, Yunshen Wang, Tengyu Liu, Sixu Yan, Wenyu Liu, Xinggang Wang, Baoxiong Jia, and Siyuan Huang. Omnitrack: General motion tracking via physics-consistent reference. *arXiv preprint arXiv:2602.23832*, 2026. URL https://arxiv.org/pdf/2602.23832.

[28] Qiayuan Liao, Takara E. Truong, Xiaoyu Huang, Yuman Gao, Guy Tevet, Koushil Sreenath, and C. Karen Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

[29] Zhengyi Luo, Ye Yuan, Tingwu Wang, Chenran Li, Sirui Chen, Fernando Castañeda, Zi-Ang Cao, Jiefeng Li, David Minor, Qingwei Ben, Xingye Da, Runyu Ding, Cyrus Hogg, Lina Song, Edy Lim, Eugene Jeong, Tairan He, Haoru Xue, Wenli Xiao, Zi Wang, Simon Yuen, Jan Kautz, Yan Chang, Umar Iqbal, Linxi "Jim" Fan, and Yuke Zhu. Sonic: Supersizing motion tracking for natural humanoid whole-body control. *arXiv preprint arXiv:2511.07820*, 2025. URL https://arxiv.org/pdf/2511.07820.

[30] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv: Arxiv-2310.12931*, 2023.

[31] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg.

Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. doi: 10.1109/LRA.2023.3270034.

[32] MuJoCo Lab Contributors. mjlab: Isaac lab api, powered by mujoco-warp, for rl and robotics research. https://github.com/mujocolab/mjlab, 2025. Accessed: 2025-11-20.

[33] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024. URL https://arxiv.org/abs/2410.13816.

[34] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[35] Chaoyi Pan, Zeji Yi, Guanya Shi, and Guannan Qu. Model-based diffusion for trajectory optimization. 2024. URL https://arxiv.org/abs/2407.01573.

[36] Haozhi Qi, Brent Yi, Mike Lambeta, Yi Ma, Roberto Calandra, and Jitendra Malik. From simple to complex skills: The case of in-hand object reorientation. *arXiv preprint arXiv:2501.05439*, 2025.

[37] Junbin Qiu, Zhengpeng Xie, Xiangda Yan, Yongjie Yang, and Yao Shu. Zeroth-order optimization is secretly single-step policy optimization. *arXiv preprint arXiv:2506.14460*, 2025.

[38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[39] Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. 2024.

[40] Jean-Pierre Sleiman, Farbod Farshidian, and Marco Hutter. Versatile multicontact planning and control for legged loco-manipulation. *Science Robotics*, 8(81):eadg5014, 2023. doi: 10.1126/scirobotics.adg5014.

[41] Prasanna Sriganesh, Barath Satheeshkumar, Anushree Sabnis, and Matthew Travers. Action-informed estimation and planning: Clearing clutter on staircases via quadrupedal pedipulation. *arXiv preprint arXiv:2509.20516*, 2025. doi: 10.48550/arXiv.2509.20516. URL https://arxiv.org/abs/2509.20516.

[42] H.J. Terry Suh, Tao Pang, Tong Zhao, and Russ Tedrake. Dexterous contact-rich manipulation via the contact trust region. *arXiv preprint arXiv:2505.02291*, 2025.

[43] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30. IEEE, 2017.

[44] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

[45] Kevin Tracy, John Z. Zhang, Jon Arrizabalaga, Stefan Schaal, Yuval Tassa, Tom Erez, and Zachary Manchester. The trajectory bundle method: Unifying sequential-convex programming and sampling-based trajectory optimization. *arXiv preprint arXiv:2509.26575*, 2025.

[46] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1433–1440, May 2016. doi: 10.1109/ICRA.2016.7487277.

[47] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving. *IEEE Transactions on Robotics*, 34(6):1603–1622, December 2018. ISSN 1941-0468. doi: 10.1109/TRO.2018.2865891.

[48] Lujie Yang, Xiaoyu Huang, Zhen Wu, Angjoo Kanazawa, Pieter Abbeel, Carmelo Sferrazza, C. Karen Liu, Rocky Duan, and Guanya Shi. Omniretarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction. 2025. URL https://arxiv.org/abs/2509.26633.

[49] Yuxiang Yang, Guanya Shi, Xiangyun Meng, Wenhao Yu, Tingnan Zhang, Jie Tan, and Byron Boots. Cajun: Continuous adaptive jumping using a learned centroidal controller. *arXiv preprint arXiv:2306.09557*, 2023. URL https://arxiv.org/abs/2306.09557.

[50] Zeji Yi, Chaoyi Pan, Guanqi He, Guannan Qu, and Guanya Shi. Covo-mpc: Theoretical analysis of sampling-based mpc and optimal covariance design. 2024.

[51] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. *Arxiv preprint arXiv:2306.08647*, 2023.

[52] John Z. Zhang, Shuo Yang, Gengshan Yang, Arun L. Bishop, Swaminathan Gurumurthy, Deva Ramanan, and Zachary Manchester. Slomo: A general system for legged robot motion imitation from casual videos. 2023. URL https://ieeexplore.ieee.org/abstract/document/10246373.

[53] John Z. Zhang, Taylor A. Howell, Zeji Yi, Chaoyi Pan, Guanya Shi, Guannan Qu, Tom Erez, Yuval Tassa, and Zachary Manchester. Whole-body model-predictive control of legged robots with mujoco. 2025. URL https://arxiv.org/abs/2503.04613.

[54] Wei Zhang, Han Wang, Carsten Hartmann, Marcus Weber, and Christof Schütte. Applications of the cross-entropy method to importance sampling and optimal control of diffusions. *SIAM Journal on Scientific Computing*, 36(6):A2654–A2672, 2014. doi: 10.1137/14096493X. URL https://doi.org/10.1137/14096493X.

[55] Xinghao Zhu, Yuxin Chen, Lingfeng Sun, Farzad Niroui, Simon Le Cleac'h, Jiuguang Wang, and Kuan Fang. Relic: Versatile loco-manipulation through flexible interlimb coordination. *arXiv preprint arXiv:2506.07876*, 2025.

APPENDIX

A. Task Rewards

Here, we provide a detailed description of the task rewards from the demonstration section as deployed on the Spot robot on hardware and G1 robot in simulation. Please refer to the code for an exact implementation.

All task costs below share a common template: a task-objective term (orientation alignment or goal reaching), end-effector proximity terms, body-positioning terms, control regularization, and safety terms. Object-specific grasp and approach locations are fixed sites expressed in the object frame (e.g., the left and right grasp points on the crowd barrier); they move rigidly with the estimated object pose, and no online contact-point optimization is performed. Similarly, the desired robot positions $\mathbf{p}^{\text{des}}$ are computed analytically from the current object state; we give the box-push torso target below as a representative example. Control-regularization terms act on the high-level planner action $\mathbf{a}$, not on joint-level controls.

Safety cost. All tasks share the safety cost

$$J_{\text{safety}} = w_{\text{fall}} \cdot \mathbb{1}\left[\min_t h^t_{\text{torso}} < h_{\min}\right], \tag{8}$$

which penalizes any rollout in which the robot's torso (pelvis) height drops below a threshold at any point along the prediction horizon, with $w_{\text{fall}} = 2500$ and $h_{\min} = 0.35\,\text{m}$ for Spot and $0.6\,\text{m}$ for the G1. Individual tasks add analogous indicator penalties for task-specific failures, e.g., the object tipping over during dragging or the robot leaving the hardware workspace.

Grasp cost. Tasks that require grasping (barrier upright, barrier drag, and tire rack drag) detect grasps from gripper-closure resistance. Let $e = q_{\text{grip}} - c_{\text{grip}}$ denote the difference between the measured and commanded gripper joint positions, which is positive when an object physically blocks the gripper from closing. A rollout state is labeled as grasping when $e > 0.15\,\text{rad}$ and the gripper is not fully closed, yielding

$$\begin{aligned} J_{\text{grasp}} = & -w_{\text{quality}} \cdot \mathbb{1}\left[\text{grasping}\right] \cdot \min\left(e/0.5,\, 1\right) \\ & + w_{\text{false}} \cdot \mathbb{1}\left[\text{gripper fully closed in free space}\right], \end{aligned} \tag{9}$$

with $w_{\text{quality}} = 100$ and $w_{\text{false}} = 1000$: the first term rewards firm grasps in proportion to the sensed closure resistance, and the second penalizes closing the gripper on nothing, preventing the planner from exploiting false grasps during rollouts.

1) Spot Tasks:

Tire Upright: The tire upright task uses a cost function with proximity terms to guide the robot's end-effectors toward the object, an orientation term to upright the tire, and regularization terms:

$$\begin{aligned} J_{\text{Tire Upright}} = & \, w_{\text{orient}} \cdot \exp\left(|\mathbf{y}_{\text{tire}}{}^z|/\sigma\right) \\ & + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}^{\text{des}}_{\text{gripper}}\|_2 \\ & + w_{\text{foot}} \cdot \min\left(\|\mathbf{p}_{\text{fr}} - \mathbf{p}^{\text{des}}_{\text{fr}}\|_2, \|\mathbf{p}_{\text{fl}} - \mathbf{p}^{\text{des}}_{\text{fl}}\|_2\right) \\ & + w_{\text{torso}} \cdot \|\mathbf{p}_{\text{torso}} - \mathbf{p}^{\text{des}}_{\text{torso}}\|_2 \\ & + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{safety}} \end{aligned} \tag{10}$$

where $\mathbf{y}_{\text{tire}}{}^z$ is the z-component of the tire's y-axis (zero when upright), σ is a smoothing parameter, $\mathbf{p}_{\text{gripper}}$, $\mathbf{p}_{\text{fr}}$, $\mathbf{p}_{\text{fl}}$, and $\mathbf{p}_{\text{torso}}$ are the positions of the gripper, front-right foot, front-left foot, and torso, respectively. The desired positions $\mathbf{p}^{\text{des}}$ are computed dynamically based on the tire position, encouraging the robot to position itself around the tire for manipulation. Each desired position is a fixed offset expressed in the tire frame—a grasp point on the tire for the gripper, contact points for the feet, and a standoff pose for the torso—so that all targets move rigidly with the estimated tire pose. J_{safety} includes penalties for the robot falling.

Crowd Barrier Upright: The crowd barrier upright task uses orientation alignment, grasp-aware proximity terms, and grasp quality feedback:

$$\begin{aligned} J_{\text{Barrier}} = & \, w_{\text{orient}} \cdot \left(1 - \exp\left(\alpha \cdot (\mathbf{z}_{\text{barrier}} \cdot \mathbf{z}_{\text{world}} - 1)\right)\right) \\ & + w_{\text{grasp}} \cdot \min\left(\|\mathbf{p}_{\text{grip}} - \mathbf{p}^{L}_{\text{grasp}}\|, \|\mathbf{p}_{\text{grip}} - \mathbf{p}^{R}_{\text{grasp}}\|\right) \\ & + w_{\text{grip}} \cdot \left(1 - |\mathbf{x}_{\text{grip}} \cdot \mathbf{x}_{\text{barrier}}| + 1 - |\mathbf{y}_{\text{grip}} \cdot \mathbf{z}_{\text{barrier}}|\right) \\ & + w_{\text{approach}} \cdot \min\left(\|\mathbf{p}_{\text{torso}} - \mathbf{p}^{L}_{\text{appr}}\|, \|\mathbf{p}_{\text{torso}} - \mathbf{p}^{R}_{\text{appr}}\|\right) \\ & + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{grasp}} + J_{\text{safety}} \end{aligned} \tag{11}$$

where $\mathbf{z}_{\text{barrier}}$ is the barrier's z-axis (aligned with $\mathbf{z}_{\text{world}}$ when upright), α is a sparsity parameter, and $\mathbf{p}^{L/R}_{\text{grasp}}$ are left/right grasp points on the barrier. The gripper orientation cost aligns the gripper's x-axis with the barrier's long axis and the gripper's y-axis with the barrier's z-axis. J_{grasp} rewards successful grasps detected via gripper resistance (position error when closing) and penalizes closing the gripper in empty space, as formalized at the beginning of this appendix. J_{safety} penalizes robot falling. The weights are w_{orient}=100 (with α=3), w_{grasp}=500, w_{grip}=100, w_{approach}=10, w_{vel}=10, and w_{ctrl}=2.

Traffic Cone Upright: The traffic cone upright task uses orientation alignment and proximity terms:

$$\begin{aligned} J_{\text{Cone}} = & \, w_{\text{orient}} \cdot \left(1 - \exp\left(\alpha \cdot (\mathbf{z}_{\text{cone}} \cdot \mathbf{z}_{\text{world}} - 1)\right)\right) \\ & + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{cone}}\|_2 \\ & - w_{\text{torso}} \cdot \min\left(d_{\text{thresh}}, \|\mathbf{p}_{\text{torso}} - \mathbf{p}_{\text{cone}}\|_2\right) \\ & + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{safety}} \end{aligned} \tag{12}$$

where $\mathbf{z}_{\text{cone}}$ is the cone's z-axis (aligned with $\mathbf{z}_{\text{world}}$ when upright), and d_{thresh} is a threshold that caps the torso term. Note the sign of the torso term: because it enters the cost negatively, it *rewards* torso distance from the cone up to the threshold d_{thresh}, encouraging the body to keep a standoff from the object while the gripper term draws the end-effector in. This division of labor—end-effector close, body clear—prevents the base from crowding or trampling the object while the arm manipulates it. J_{safety} includes penalties for robot falling.

Chair Upright: The chair upright task uses a similar cost structure as the traffic cone:

$$\begin{aligned} J_{\text{Chair}} = w_{\text{orient}} \cdot (1 - \exp(\alpha \cdot (\mathbf{z}_{\text{chair}} \cdot \mathbf{z}_{\text{world}} - 1))) \\ + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{chair}}\|_2 \\ - w_{\text{torso}} \cdot \min(d_{\text{thresh}}, \|\mathbf{p}_{\text{torso}} - \mathbf{p}_{\text{chair}}\|_2) \\ + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{safety}} \end{aligned} \tag{13}$$

where $\mathbf{z}_{\text{chair}}$ is the chair's z-axis (aligned with $\mathbf{z}_{\text{world}}$ when upright), $\mathbf{p}_{\text{gripper}}$ and $\mathbf{p}_{\text{chair}}$ are the gripper and chair positions, and d_{thresh} is a threshold that caps the torso term. As in the cone task, the negative torso term rewards a body standoff of up to d_{thresh} from the chair, keeping the base clear while the gripper term attracts the end-effector. $\mathbf{v}_{\text{obj}}$ penalizes fast object movements, $\mathbf{a}$ is the planner action, and J_{safety} includes penalties for robot falling.

Tire Stack: The tire stack task requires placing a standing tire on top of a flat tire, using stacking alignment and proximity terms:

$$\begin{aligned} J_{\text{Stack}} = w_{xy} \cdot \|\mathbf{p}_{\text{top}}^{xy} - \mathbf{p}_{\text{bottom}}^{xy}\|_2 + w_z \cdot |p_{\text{top}}^z - p_{\text{desired}}^z| \\ + w_{\text{orient}} \cdot (1 - \mathbf{y}_{\text{top}} \cdot \hat{\mathbf{u}}_{\text{stack}}) \\ + w_{\text{bottom}} \cdot (\|\mathbf{v}_{\text{bottom}}\|_2 + \|\boldsymbol{\omega}_{\text{bottom}}\|_2) \\ + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{gripper}}^{\text{des}}\|_2 \\ + w_{\text{torso}} \cdot \|\mathbf{p}_{\text{torso}} - \mathbf{p}_{\text{torso}}^{\text{des}}\|_2 + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{safety}} \end{aligned} \tag{14}$$

where $\mathbf{p}_{\text{top}}^{xy}$ and $\mathbf{p}_{\text{bottom}}^{xy}$ are the XY positions of the top and bottom tires (top should be directly above bottom), p_{desired}^z is the target height for proper stacking, $\mathbf{y}_{\text{top}}$ is the top tire's y-axis, and $\hat{\mathbf{u}}_{\text{stack}}$ is the unit vector from top to bottom tire. The bottom tire velocity term keeps the target tire stationary. Desired gripper and torso positions are computed dynamically based on the stacking geometry. The weights are w_{xy}=100, w_z=200, w_{orient}=20, w_{bottom}=50, w_{gripper}=10, w_{torso}=10, and w_{ctrl}=1; the orientation term is activated only once the top tire is within 2 m of the stacking target.

Crowd Barrier Drag: The crowd barrier drag task extends the barrier upright cost with a goal-reaching term:

$$\begin{aligned} J_{\text{Barrier Drag}} = w_{\text{goal}} \cdot \|\mathbf{p}_{\text{barrier}} - \mathbf{p}_{\text{goal}}\|_2 \\ + w_{\text{orient}} \cdot (1 - \exp(\alpha \cdot (\mathbf{z}_{\text{barrier}} \cdot \mathbf{z}_{\text{world}} - 1))) \\ + w_{\text{grasp}} \cdot \min\left(\|\mathbf{p}_{\text{grip}} - \mathbf{p}_{\text{grasp}}^L\|, \|\mathbf{p}_{\text{grip}} - \mathbf{p}_{\text{grasp}}^R\|\right) \\ + w_{\text{grip}} \cdot (1 - |\mathbf{x}_{\text{grip}} \cdot \mathbf{x}_{\text{barrier}}| + 1 - |\mathbf{y}_{\text{grip}} \cdot \mathbf{z}_{\text{barrier}}|) \\ + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{grasp}} + J_{\text{safety}} \end{aligned} \tag{15}$$

where $\mathbf{p}_{\text{goal}}$ is the target position. The weights are w_{goal}=500, w_{orient}=50, w_{grasp}=250, w_{grip}=100, w_{vel}=10, and w_{ctrl}=2; J_{safety} additionally penalizes the barrier tipping over during the drag.

Tire Rack Drag: The tire rack drag task uses goal-reaching, orientation alignment, and grasp terms:

$$\begin{aligned} J_{\text{Rack}} = w_{\text{goal}} \cdot \|\mathbf{p}_{\text{rack}} - \mathbf{p}_{\text{goal}}\|_2 \\ + w_{\text{orient}} \cdot \big((1 - \mathbf{x}_{\text{rack}} \cdot \mathbf{x}_{\text{world}}) \\ + (1 - \mathbf{y}_{\text{rack}} \cdot \mathbf{y}_{\text{world}}) \\ + (1 - \mathbf{z}_{\text{rack}} \cdot \mathbf{z}_{\text{world}})\big) \\ + w_{\text{grasp}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{grasp}}\|_2 \\ + w_{\text{grip}} \cdot (1 - \mathbf{z}_{\text{gripper}} \cdot \mathbf{z}_{\text{rack}}) \\ + w_{\text{approach}} \cdot \min_i \|\mathbf{p}_{\text{torso}} - \mathbf{p}_{\text{appr}}^i\|_2 + J_{\text{grasp}} + J_{\text{safety}} \end{aligned} \tag{16}$$

where the orientation cost keeps the rack aligned with world axes, $\mathbf{p}_{\text{appr}}^i$ are multiple approach sites (left, mid, right), and the gripper orientation cost aligns the gripper's z-axis with the rack's z-axis. J_{safety} includes penalties for robot falling and rack tipping over. The weights are w_{goal}=500, w_{orient}=50, w_{grasp}=250, w_{grip}=250, and w_{approach}=100.

Rugged Box Push: The rugged box push task uses goal-reaching with dynamic torso positioning for pushing:

$$\begin{aligned} J_{\text{Box Push}} = w_{\text{goal}} \cdot \|\mathbf{p}_{\text{box}} - \mathbf{p}_{\text{goal}}\|_2 \\ + w_{\text{orient}} \cdot \big(|1 - \mathbf{x}_{\text{box}} \cdot \mathbf{x}_{\text{world}}| \\ + |1 - \mathbf{y}_{\text{box}} \cdot \mathbf{y}_{\text{world}}| + |1 - \mathbf{z}_{\text{box}} \cdot \mathbf{z}_{\text{world}}|\big) \\ + w_{\text{torso}} \cdot \|\mathbf{p}_{\text{torso}} - \mathbf{p}_{\text{torso}}^{\text{des}}\|_2 \\ + w_{\text{gripper}} \cdot \|\mathbf{p}_{\text{gripper}} - \mathbf{p}_{\text{box}}\|_2 \\ + w_{\text{ctrl}} \cdot \|\mathbf{a}\|_2 + J_{\text{safety}} \end{aligned} \tag{17}$$

where the desired torso position $\mathbf{p}_{\text{torso}}^{\text{des}}$ is computed dynamically as the position opposite the goal direction from the box, encouraging the robot to position behind the box for pushing. Specifically, in the horizontal plane, $\mathbf{p}_{\text{torso}}^{\text{des}} = \mathbf{p}_{\text{box}} - d \cdot (\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{box}}) / \|\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{box}}\|$ with standoff $d = 0.6$ m and a fixed desired torso height of 0.4 m, which places the robot behind the box relative to the goal. The weights are w_{goal}=500, w_{orient}=50, w_{torso}=30, w_{gripper}=0.1, and w_{ctrl}=2.

2) G1 Tasks:

Box Pushing: The G1 box pushing task uses goal-reaching, orientation, and bimanual proximity terms:

$$\begin{aligned} J_{\text{G1 Box}} = w_{\text{goal}} \cdot \|\mathbf{p}_{\text{box}} - \mathbf{p}_{\text{goal}}\|_2 + w_{\text{orient}} \cdot |1 - \mathbf{y}_{\text{box}} \cdot \mathbf{z}_{\text{world}}| \\ + w_{\text{hand}} \cdot \min(\|\mathbf{p}_{\text{left}} - \mathbf{p}_{\text{box}}\|, \|\mathbf{p}_{\text{right}} - \mathbf{p}_{\text{box}}\|) \\ - w_{\text{pelvis}} \cdot \|\mathbf{p}_{\text{pelvis}} - \mathbf{p}_{\text{box}}\|_2 - w_{\text{facing}} \cdot \mathbf{x}_{\text{robot}} \cdot \mathbf{x}_{\text{world}} \\ + w_{\text{ctrl}} \cdot (\|\mathbf{v}_{\text{base}}\|_2 + \|\mathbf{q}_{\text{arm}} - \mathbf{q}_{\text{arm}}^{\text{default}}\|_2) + J_{\text{safety}} \end{aligned} \tag{18}$$

where $\mathbf{y}_{\text{box}}$ is the box's y-axis (should point up), $\mathbf{p}_{\text{left/right}}$ are left/right palm positions, and the robot orientation term encourages facing forward. The control cost penalizes base velocity and arm deviation from default poses. The negative pelvis term (with a small weight) rewards keeping the torso from crowding the box, so that the hands, rather than the body, make contact; the same term appears in the remaining G1 tasks. The weights are w_{goal}=50, w_{orient}=15, w_{hand}=10, w_{pelvis}=0.1, w_{facing}=50, and w_{ctrl}=3.

Chair Pushing: The G1 chair pushing task is similar to box pushing with XY-only goal distance:

$$\begin{aligned} J_{\text{G1 Chair}} = & \, w_{\text{goal}} \cdot \|\mathbf{p}_{\text{chair}}^{xy} - \mathbf{p}_{\text{goal}}^{xy}\|_2 + w_{\text{orient}} \cdot |1 - \mathbf{z}_{\text{chair}} \cdot \mathbf{z}_{\text{world}}| \\ & + w_{\text{hand}} \cdot \min\left(\|\mathbf{p}_{\text{left}} - \mathbf{p}_{\text{chair}}\|, \|\mathbf{p}_{\text{right}} - \mathbf{p}_{\text{chair}}\|\right) \\ & - w_{\text{pelvis}} \cdot \|\mathbf{p}_{\text{pelvis}} - \mathbf{p}_{\text{chair}}\|_2 + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 \\ & + w_{\text{ctrl}} \cdot (\|\mathbf{v}_{\text{base}}\|_2 + \|\mathbf{q}_{\text{arm}} - \mathbf{q}_{\text{arm}}^{\text{default}}\|_2) + J_{\text{safety}} \end{aligned} \tag{19}$$

where the goal distance uses only the XY components, and $\mathbf{z}_{\text{chair}}$ is the chair's z-axis (should point up). An additional object velocity term penalizes fast chair movements. The weights are w_{goal}=30, w_{orient}=15, w_{hand}=10, w_{pelvis}=0.1, w_{vel}=10, and w_{ctrl}=3.

Door Opening: The G1 door opening task uses goal-reaching (to the other side of the door) and hand-to-handle proximity:

$$\begin{aligned} J_{\text{G1 Door}} = & \, w_{\text{goal}} \cdot \|\mathbf{p}_{\text{pelvis}} - \mathbf{p}_{\text{goal}}\|_2 + w_{\text{hand}} \cdot \|\mathbf{p}_{\text{right}} - \mathbf{p}_{\text{handle}}\|_2 \\ & - w_{\text{pelvis}} \cdot \|\mathbf{p}_{\text{pelvis}} - \mathbf{p}_{\text{door}}\|_2 - w_{\text{facing}} \cdot \mathbf{x}_{\text{robot}} \cdot \mathbf{x}_{\text{world}} \\ & + w_{\text{ctrl}} \cdot (\|\mathbf{v}_{\text{base}}\|_2 + \|\mathbf{q}_{\text{arm}} - \mathbf{q}_{\text{arm}}^{\text{default}}\|_2) + J_{\text{safety}} \end{aligned} \tag{20}$$

where $\mathbf{p}_{\text{goal}}$ is on the other side of the door (requiring the robot to open it), $\mathbf{p}_{\text{handle}}$ is the door handle position, and $\mathbf{p}_{\text{door}}$ is the door center. The weights are w_{goal}=30, w_{hand}=5, w_{facing}=50, and w_{ctrl}=3.

Table Pushing: The G1 table pushing task uses XY goal distance, table orientation, and bimanual proximity terms:

$$\begin{aligned} J_{\text{G1 Table}} = & \, w_{\text{goal}} \cdot \|\mathbf{p}_{\text{table}}^{xy} - \mathbf{p}_{\text{goal}}^{xy}\|_2 + w_{\text{orient}} \cdot |1 - \mathbf{y}_{\text{table}} \cdot \mathbf{z}_{\text{world}}| \\ & + w_{\text{hand}} \cdot \min\left(\|\mathbf{p}_{\text{left}} - \mathbf{p}_{\text{table}}\|, \|\mathbf{p}_{\text{right}} - \mathbf{p}_{\text{table}}\|\right) \\ & - w_{\text{pelvis}} \cdot \|\mathbf{p}_{\text{pelvis}} - \mathbf{p}_{\text{table}}\|_2 + w_{\text{vel}} \cdot \|\mathbf{v}_{\text{obj}}\|_2^2 \\ & + w_{\text{ctrl}} \cdot (\|\mathbf{v}_{\text{base}}\|_2 + \|\mathbf{q}_{\text{arm}} - \mathbf{q}_{\text{arm}}^{\text{default}}\|_2) + J_{\text{safety}} \end{aligned} \tag{21}$$

where the goal distance uses only the XY components, and $\mathbf{y}_{\text{table}}$ is the table's y-axis (should point up when upright). The weights are w_{goal}=30, w_{orient}=15, w_{hand}=10, w_{pelvis}=0.1, and w_{ctrl}=3.